

TRELLIS 2.0 API 使用说明

1. API服务概述

TRELLIS 2.0 API 是一个基于深度学习的图像到3D模型转换服务，它能够将2D图像转换为高质量的3D模型。

核心功能：

- 上传2D图像，生成对应的3D模型
- 支持多种参数调整，控制生成质量和速度
- 返回生成的视频预览和GLB格式3D模型
- 模型只加载一次，提高处理效率

技术特点：

- 使用微软开源的TRELLIS 2.0模型
- 支持多种分辨率设置
- 提供丰富的参数调整选项
- 基于FastAPI构建，性能优异

2. 访问API

服务地址

API服务通过serveo提供的临时URL访问：

`https://trellis2-suyang.serveusercontent.com`

访问方式

- 所有请求使用HTTP协议
- 图像上传使用 `multipart/form-data` 格式
- 响应格式为JSON

3. API端点说明

3.1 健康检查

- **端点:** GET /api/health
- **功能:** 检查AP服务是否正常运行
- **响应:**

```
{
  "status": "healthy",
  "message": "TRELLIS 2.0 API is running"
}
```

3.2 处理图像

- **端点:** POST /api/process
- **功能:** 上传图像并生成3D模型（异步处理）
- **参数:**

参数	类型	必填	默认值	说明
image	文件	是	-	输入图像文件 (支持JPG、PNG等格式)
resolution	字符串	否	"1024"	生成分辨率: "512", "1024", "1536"
seed	整数	否	0	随机种子
randomize_seed	布尔值	否	true	是否随机化种子
decimation_target	整数	否	500000	网格简化目标 (面数)
texture_size	整数	否	2048	纹理大小
ss_guidance_strength	浮点数	否	7.5	稀疏结构生成引导强度
ss_guidance_rescale	浮点数	否	0.7	稀疏结构生成引导重缩放
ss_sampling_steps	整数	否	12	稀疏结构生成采样步骤
ss_rescale_t	浮点数	否	5.0	稀疏结构生成重缩放T值

参数	类型	必填	默认值	说明
shape_slat_guidance_strength	浮点数	否	7.5	形状生成引导强度
shape_slat_guidance_rescale	浮点数	否	0.5	形状生成引导重缩放
shape_slat_sampling_steps	整数	否	12	形状生成采样步骤
shape_slat_rescale_t	浮点数	否	3.0	形状生成重缩放T值
tex_slat_guidance_strength	浮点数	否	1.0	材质生成引导强度
tex_slat_guidance_rescale	浮点数	否	0.0	材质生成引导重缩放
tex_slat_sampling_steps	整数	否	12	材质生成采样步骤
tex_slat_rescale_t	浮点数	否	3.0	材质生成重缩放T值

• 响应:

```
{
  "status": "processing",
  "message": "Task has been started",
  "task_id": "12345678-1234-1234-1234-1234567890ab",
  "poll_url": "/api/status/12345678-1234-1234-1234-1234567890ab"
}
```

3.3 查询任务状态

- 端点: GET /api/status/{task_id}
- 功能: 查询任务的处理状态和结果
- 参数: task_id - 任务ID (从 /api/process 响应中获取)
- 响应:

◦ 处理中:

```
{
  "status": "pending",
  "message": "Task is still processing"
}
```

◦ 处理成功:

```
{
  "status": "success",
  "message": "Processing completed successfully",
  "video_url": "/api/download/result_xxx.mp4",
  "glb_url": "/api/download/result_xxx.glb"
}
```

◦ 处理失败:

```
{
  "status": "error",
  "message": "Processing failed: error message"
}
```

3.4 下载文件

- **端点:** GET /api/download/{filename}
- **功能:** 下载生成的视频或GLB文件
- **参数:** filename - 要下载的文件名
- **响应:** 文件下载

4. 调用示例

4.1 cURL示例 (分步)

步骤1: 上传图像获取任务ID

```
curl -X POST "https://your-serveo-url/api/process" \
  -F "image=@path/to/image.png" \
  -F "resolution=1024" \
  -F "decimation_target=500000"
```

步骤2: 轮询任务状态 (每10秒运行一次, 直到返回success)

```
curl "https://your-serveo-url/api/status/任务ID"
```

步骤3: 下载结果 (使用返回的下载链接)

```
curl -O "https://your-serveo-url/api/download/result_xxx.mp4"
curl -O "https://your-serveo-url/api/download/result_xxx.glb"
```

4.2 Python示例（带轮询）

```
import requests
import time

# 步骤1: 上传图像
def upload_image():
    url = "https://your-serveo-url/api/process"
    files = {'image': open('path/to/image.png', 'rb')}
    data = {
        'resolution': '1024',
        'decimation_target': '500000'
    }

    response = requests.post(url, files=files, data=data)
    return response.json()

# 步骤2: 轮询任务状态
def poll_status(task_id):
    max_attempts = 60
    poll_interval = 10

    for attempt in range(max_attempts):
        time.sleep(poll_interval)
        status_url = f"https://your-serveo-url/api/status/{task_id}"
        response = requests.get(status_url)
        status_data = response.json()

        print(f"轮询 {attempt+1}/{max_attempts}: {status_data['status']}")

        if status_data['status'] == 'success':
            return status_data
        elif status_data['status'] == 'error':
            raise Exception(f"处理失败: {status_data['message']}")

    raise Exception("轮询超时")

# 执行流程
if __name__ == "__main__":
    print("上传图像...")
    upload_result = upload_image()
    print(f"任务ID: {upload_result['task_id']}")
```

```
print("轮询任务状态...")
result = poll_status(upload_result['task_id'])

print("处理成功!")
print(f"视频链接: https://your-serveo-url{result['video_url']}")
print(f"GLB链接: https://your-serveo-url{result['glb_url']}")
```

4.3 JavaScript示例（带轮询）

```
async function processImage(imageFile) {
  // 步骤1: 上传图像
  const formData = new FormData();
  formData.append('image', imageFile);
  formData.append('resolution', '1024');
  formData.append('decimation_target', '500000');

  const response = await fetch('https://your-serveo-url/api/process', {
    method: 'POST',
    body: formData
  });

  const uploadResult = await response.json();
  console.log('任务ID:', uploadResult.task_id);

  // 步骤2: 轮询任务状态
  const maxAttempts = 60;
  const pollInterval = 10000; // 10秒

  for (let attempt = 0; attempt < maxAttempts; attempt++) {
    await new Promise(resolve => setTimeout(resolve, pollInterval));

    const statusResponse = await fetch(`https://your-serveo-url/api/status/${uploadResult.task_id}`);
    const statusData = await statusResponse.json();

    console.log(`轮询 ${attempt+1}/${maxAttempts}:`, statusData.status);

    if (statusData.status === 'success') {
      return statusData;
    } else if (statusData.status === 'error') {
      throw new Error(`处理失败: ${statusData.message}`);
    }
  }

  throw new Error('轮询超时');
}

// 使用示例
async function main() {
  try {
    const result = await processImage(imageFile);
  } catch (error) {
    console.error(error);
  }
}
```

```
    console.log('处理成功!');
    console.log('视频链接:', `https://your-serveo-url${result.video_url}`);
    console.log('GLB链接:', `https://your-serveo-url${result.glb_url}`);
  } catch (error) {
    console.error('错误:', error);
  }
}
```


4.4 Windows批处理示例

```
@echo off
setlocal enabledelayedexpansion

:: 设置API地址
set "API_URL=https://your-serveo-url/api/process"
set "STATUS_BASE_URL=https://your-serveo-url/api/status"

:: 设置参数
set "IMAGE_PATH=E:\path\to\image.png"
set "RESOLUTION=1024"
set "DECIMATION_TARGET=500000"

:: 上传图像
for /f "delims=" %%i in ('curl -s -X POST "%API_URL%" -F "image=@%IMAGE_PATH%" -F "resolu'

:: 提取task_id
for /f "tokens=2 delims=":", " %%a in ('echo %RESPONSE% ^| findstr "task_id"') do set "TAS

:: 轮询状态
set "MAX_ATTEMPTS=60"
set "POLL_INTERVAL=10"
set "ATTEMPT=0"

:POLL_LOOP
set /a ATTEMPT+=1
if %ATTEMPT% gtr %MAX_ATTEMPTS% goto :TIMEOUT

ping 127.0.0.1 -n %POLL_INTERVAL% > nul

for /f "delims=" %%i in ('curl -s "%STATUS_BASE_URL%/TASK_ID%") do set "STATUS_RESPONSE:

echo 轮询 %ATTEMPT%/MAX_ATTEMPTS%: !STATUS_RESPONSE!

echo !STATUS_RESPONSE! | findstr "success" > nul
if %errorlevel% equ 0 goto :SUCCESS

echo !STATUS_RESPONSE! | findstr "error" > nul
if %errorlevel% equ 0 goto :ERROR

goto :POLL_LOOP
```

:SUCCESS

echo 处理成功!

for /f "tokens=2 delims=":",'" %%a in ('echo !STATUS_RESPONSE! ^| findstr "video_url"') do

for /f "tokens=2 delims=":",'" %%a in ('echo !STATUS_RESPONSE! ^| findstr "glb_url"') do s

echo 视频: https://your-serveo-url!VIDEO_URL!

echo GLB: https://your-serveo-url!GLB_URL!

goto :END

:ERROR

echo 处理失败: !STATUS_RESPONSE!

goto :END

:TIMEOUT

echo 轮询超时

goto :END

:END

pause

4.5 PowerShell示例

```
# 设置参数
$apiUrl = "https://your-serveo-url/api/process"
$imagePath = "E:\path\to\image.png"

# 上传图像
Write-Host "上传图像..."
$form = @{
    image = Get-Item $imagePath
    resolution = "1024"
    decimation_target = "500000"
}

$response = Invoke-RestMethod -Uri $apiUrl -Method Post -Form $form
Write-Host "任务ID: $($response.task_id)"

# 轮询状态
Write-Host "轮询任务状态..."
$maxAttempts = 60
$pollInterval = 10

for ($i = 1; $i -le $maxAttempts; $i++) {
    Start-Sleep -Seconds $pollInterval
    $statusUrl = "https://your-serveo-url/api/status/$($response.task_id)"
    $statusResponse = Invoke-RestMethod -Uri $statusUrl

    Write-Host "轮询 $i/$maxAttempts: $($statusResponse.status)"

    if ($statusResponse.status -eq "success") {
        Write-Host "处理成功!"
        Write-Host "视频链接: https://your-serveo-url/$($statusResponse.video_url)"
        Write-Host "GLB链接: https://your-serveo-url/$($statusResponse.glb_url)"
        break
    } elseif ($statusResponse.status -eq "error") {
        Write-Host "处理失败: $($statusResponse.message)"
        break
    }
}
```

5. 注意事项

5.1 服务稳定性

- **临时服务**：API服务基于serveo提供的临时隧道，可能会因网络波动而断开
- **异步处理**：API采用异步处理机制，避免了serveo 502超时错误，提高了服务稳定性

5.2 处理时间

- **处理时长**：生成复杂的3D模型可能需要较长时间（数分钟）
- **超时设置**：客户端应设置适当的超时时间（建议至少90秒）
- **异步处理**：对于前端应用，建议使用异步处理方式，避免页面卡顿
- **轮询机制**：使用轮询方式获取处理结果，避免长时间等待导致的连接超时

5.3 文件大小限制

- **图像大小**：建议图像大小不超过10MB
- **输出文件**：生成的GLB文件大小可能在几十MB到几百MB之间
- **网络带宽**：大文件下载可能受网络带宽限制

5.5 资源消耗

- **并发限制**：高分辨率生成需要大量GPU内存，处理过程会占用大量CPU资源，建议不要同时处理多个高分辨率请求

5.6 错误处理

- **网络错误**：处理网络波动导致的连接失败
- **参数错误**：确保提供正确的参数格式
- **文件错误**：确保上传的文件是有效的图像格式
- **502 Bad Gateway**：使用轮询机制获取结果，避免serveo超时错误
- **轮询超时**：如果超过轮询次数仍未完成，任务可能仍在处理中，可稍后再次查询

6. 故障排除

6.1 常见错误及解决方案

错误代码	可能原因	解决方案
400 Bad Request	参数错误或文件格式不正确	检查参数格式和文件类型
500 Internal Server Error	服务器内部错误	联系管理员检查服务器日志，重启服务
502 Bad Gateway	serveo隧道连接问题	联系管理员重新启动serveo隧道
504 Gateway Timeout	处理时间过长	增加超时设置，或降低分辨率

6.2 调试建议

- 1. **测试健康检查**：首先测试 `/api/health` 端点，确认服务正常运行
- 2. **检查服务日志**：查看API服务的输出日志，了解具体错误原因
- 3. **简化测试**：使用小图像和低分辨率进行测试，排除资源问题
- 4. **网络测试**：使用curl命令测试API，排除客户端代码问题
- 5. **重启服务**：如果遇到持续错误，联系管理员尝试重启API服务和serveo隧道

7. 示例应用

7.1 HTML表单示例

```
<!DOCTYPE html>
<html>
<head>
  <title>TRELLIS 2.0 API 测试</title>
</head>
<body>
  <h1>上传图像生成3D模型</h1>
  <form id="uploadForm">
    <input type="file" id="image" name="image" accept="image/*" required>
    <br><br>
    <label>分辨率:</label>
    <select name="resolution">
      <option value="512">512</option>
      <option value="1024" selected>1024</option>
      <option value="1536">1536</option>
    </select>
    <br><br>
    <button type="submit">生成3D模型</button>
  </form>
  <div id="result"></div>

  <script>
    document.getElementById('uploadForm').addEventListener('submit', async function(e) {
      e.preventDefault();
      const formData = new FormData(this);
      const resultDiv = document.getElementById('result');
      resultDiv.innerHTML = '处理中...';

      try {
        const response = await fetch('https://your-serveo-url/api/process', {
          method: 'POST',
          body: formData
        });

        if (!response.ok) throw new Error('API请求失败');

        const data = await response.json();
        resultDiv.innerHTML = `
```

```
        <h2>处理成功!</h2>
        <p><a href="${data.video_url}" target="_blank">下载视频</a></p>
        <p><a href="${data.glb_url}" target="_blank">下载GLB文件</a></p>
    `;
    } catch (error) {
        resultDiv.innerHTML = `错误: ${error.message}`;
    }
    });
</script>
</body>
</html>
```

7.2 批量处理脚本示例

```
import os
import requests

def process_images(image_dir, output_dir):
    """批量处理目录中的所有图像"""
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    api_url = "https://your-serveo-url/api/process"

    for filename in os.listdir(image_dir):
        if filename.endswith(('.png', '.jpg', '.jpeg')):
            image_path = os.path.join(image_dir, filename)
            print(f"处理图像: {filename}")

            try:
                files = {'image': open(image_path, 'rb')}
                data = {
                    'resolution': '1024',
                    'decimation_target': '500000',
                    'texture_size': '2048'
                }

                response = requests.post(api_url, files=files, data=data, timeout=60)
                if response.status_code == 200:
                    result = response.json()
                    print(f"成功: {result['message']}")
                    print(f"视频链接: {result['video_url']}")
                    print(f"GLB链接: {result['glb_url']}")
                else:
                    print(f"失败: {response.status_code}")
            except Exception as e:
                print(f"错误: {e}")
            print("-" * 50)

# 使用示例
if __name__ == "__main__":
    process_images('input_images', 'output_results')
```


8. 联系与支持

如果您在使用API过程中遇到任何问题，请联系服务管理员：

- 管理员： suyang.li
- 服务器： fduvis
- 邮箱： 24300980051@m.fudan.edu.cn

注意：本API服务仅供开发和测试使用，生产环境部署需要考虑安全性、稳定性和可扩展性。